

Enhancing Log Monitoring with ELK, LLM-Based Analysis, Automated Alerts, and Self-Healing Mechanisms

Varad Pundlik

Dept. of Computer Engineering, SCTR's
Pune Institute of Computer
Technology,
Pune, India
varadpundlik@gmail.com

Anirudha Udgirkar

Dept. of Computer Engineering, SCTR's
Pune Institute of Computer
Technology,
Pune, India
udgirkaranirudha@gmail.com

Divyank Sagvekar

Dept. of Computer Engineering, SCTR's
Pune Institute of Computer
Technology,
Pune, India
divyanksagvekar37@gmail.com

Parth Tagalpallewar

Dept. of Computer Engineering, SCTR's
Pune Institute of Computer
Technology,
Pune, India
parthtagalpallewar123@gmail.com

Snehal Shintre

Assistant Professor, Dept. of Computer
Engineering, SCTR's Pune Institute of
Computer Technology,
Pune, India
spshintre@pict.edu

Abstract—Managing and analyzing application logs is essential for debugging, performance monitoring, and security. Traditional log monitoring methods often rely on native scripting or standalone software, limiting scalability and real-time processing. This paper presents an implementation of a robust Logs Management System utilizing Elastic Search, Filebeat and Metricbeat for efficient log collection. The proposed system further incorporates a log classification system for structured log analysis, an LLM-based log analysis framework for intelligent insights, and an automated alert generation mechanism coupled with self-healing capabilities. Through experimentation and real-world testing, improved anomaly detection and enhanced automation in log management is demonstrated.

Keywords— Log analysis, Elastic Search, Filebeat and Metricbeat, LLM-based log analysis, Automated Alerting, Self-healing systems, Real-time monitoring

I. INTRODUCTION

With the exponential growth of distributed systems, logging has become an integral part of application monitoring, debugging, and security enforcement. Logs store crucial system interactions, errors, and performance metrics. Traditional methods for log collection and analysis, such as shell scripting and ad-hoc log parsing, struggle to scale with increasing log volumes. The Elastic Search, Filebeat and Metricbeat provides a scalable and efficient solution for log management, enabling real-time monitoring and visualization.

This paper introduces an advanced implementation of a log management system that integrates ELK with AI-powered log analysis, automated alerts, and self-healing mechanisms. This dashboard include features:

1. A log classification system to categorize logs and extract relevant insights.
2. LLM-based log analysis for anomaly detection and predictive monitoring.
3. Automated alert generation and self-healing mechanisms for proactive incident management.

II. LITERATURE REVIEW

The field of log management has evolved significantly, incorporating advanced frameworks such as ELK, Splunk, and cloud-native solutions. Researchers have explored automated log classification, anomaly detection, and root cause analysis using machine learning techniques. This section examines various methodologies, comparing their effectiveness.

A. ELK-based Distributed Log Collection

The ELK stack (Elasticsearch, Logstash, Kibana) is a widely used open-source log management system that provides distributed log collection and real-time analytics. Research by Zong et al. [1] integrates Apache Flume and Kafka to create a scalable log collection framework that standardizes log formats and efficiently manages large-scale log data. The system utilizes Redis and Kafka as message brokers, storing logs in Elasticsearch for quick retrieval and visualization via Kibana.

B. Splunk-based Log Collection

Splunk is another widely used log management solution that focuses on centralized log aggregation. Okumura et al. [2] propose a Splunk-based logging system for university networking environments. The system collects logs using network syslog and Splunk forwarder, offering an indexed and searchable dataset for network administrators. While effective, Splunk's proprietary nature limits customization compared to ELK.

C. Cloud-Native Log Collection in Kubernetes

With the rise of cloud computing, Kubernetes-based log management has gained significance. Chen et al. [3] develop a cloud-native log collection system that integrates with Kubernetes to manage logs across containerized environments. This approach captures logs from standard output (stdout) and standard error (stderr) streams, preventing excessive disk usage through log rotation techniques.

D. Real-Time System Log Monitoring

Gunasekaran et al. [4] introduce a real-time monitoring framework for supercomputers, aggregating logs from multiple sources and processing them dynamically to detect anomalies. The system enables administrators to analyze live data streams, reducing downtime and improving performance optimization. However, challenges remain in handling high computational demands and detecting anomalies in first-time occurrences.

E. Automated Log Classification Using Deep Learning

Ramachandran et al. [5] propose a deep learning-based framework for log classification and anomaly detection. Using a hybrid CNN-LSTM architecture, their model achieves 99.19% accuracy in classifying log messages. The approach significantly reduces manual intervention and enhances the efficiency of log analysis in large-scale enterprise applications.

F. Monitoring Access Log Database of information system on a Database Server

In the realm of distributed log collection and monitoring, Setiyadi and Setiawan [6] introduce a system specifically designed to monitor access to a MySQL database server, with a focus on enhancing data security through real-time user access tracking. The system employs a web-based interface constructed using PHP and MySQL. Unlike general purpose log monitoring frameworks like the ELK stack this system narrows its focus on database security at the individual server level. It offers real-time insights that help prevent unauthorized access and mitigate the risk of privilege abuse.

G. Automated Log Analysis and Parsing

Automated log analysis plays a vital role in managing the massive volume of logs generated by enterprise systems. He et al. [7] review various log analysis techniques, including log compression, parsing, and anomaly detection using clustering and deep learning. Their findings highlight the importance of scalable log processing systems to maintain real-time performance monitoring capabilities. Chabridon et al. [9] propose improvements in log mining by enhancing performance through optimized log storage and indexing techniques. Lyu et al. [10] discuss the use of automated log parsing tools to enhance log structuring and retrieval, further improving the efficiency of log-based diagnostics.

H. Root Cause Analysis Using Large Language Models (LLMs)

Chen et al. [8] explore the use of LLMs for root cause analysis in cloud incidents. Their system integrates log data, system metrics, and telemetry data to identify root causes of failures. By leveraging machine learning models trained on historical data, the system predicts potential issues and recommends resolutions, reducing the need for manual troubleshooting.

I. LLM-based log analysis and parsing

Akhtar Sirraaj et al. [11] provides survey on using different LLM based techniques like prompt engineering and fine tuning to analyze logs and perform root cause analysis, anomaly, fault and threat detection on event logs

Wei Guan et al. [12] uses fine tuning techniques on BERT and LLAMA models for anomaly classification. Since fine tuning is time and resource consuming operation and less dynamic

Recent advancements emphasize AI-driven observability frameworks. Baqar et al. [13] proposed a self-healing software architecture using continuous learning for autonomous remediation. Arora et al. [14] developed an AI-assisted cloud-monitoring system integrating LLMs for predictive maintenance.

J. Conclusion of Literature Review

The reviewed literature highlights the strengths and limitations of various log management approaches. The ELK stack remains a dominant open-source solution for real-time log monitoring and analysis. This implementation builds upon these insights by integrating ELK with AI-driven log analysis, automated alerts, and self-healing mechanisms for enhanced operational resilience.

III. METHODOLOGY

The log management dashboard is responsible for collecting logs and metrics on a real time basis, separating and summarizing logs based on distinct API operations, root cause analysis of an incident occurring, generating alerts for failures and self-healing automation to recover server and application from failure. The implementation of all these features is divided into 3 subsystems:

1. Logs and metric collection system
2. LLM based logs analysis system
3. Alert generation and self-healing automation

A. Logs and metrics collection system

The logs and metrics collection system forms the backbone of our Logs Management Dashboard. It is designed to make sure that both server logs and critical server metrics are captured, enhanced, and stored in real time, thereby providing a clear view of system performance and providing proactive error detection and resolution.

Log Collection with Filebeat:

Filebeat is a lightweight, open-source log shipper installed on every target server to continuously monitor and collect logs from specified directories. It ensures real-time log forwarding, enabling quick detection of issues. These logs are then sent to an Elasticsearch index as shown in Fig 1, where they are structured for efficient searching and further processing. This seamless integration ensures that logs are not only collected but also made actionable for monitoring and troubleshooting

Metrics Collection with Metricbeat:

Metricbeat is used to collect real-time server performance metrics, including CPU usage, memory consumption, network activity, and disk I/O. These metrics are gathered at regular intervals and sent to Elasticsearch, where they are stored in time-series indices for efficient analysis as shown in Fig1. By correlating system performance data with application logs, Metricbeat helps identify patterns and potential issues, such as performance drops leading to errors. This structured approach enables proactive monitoring, faster troubleshooting, and improved system reliability

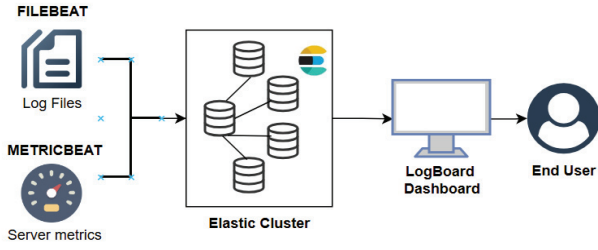


Fig. 1. System architecture of logs and metrics collection system

B. LLM based log analysis

Log analysis features consist of Operation based log separation where log blocks are partitioned into operations, API requests and transactions where the operations and their summary is also provided based on logs.

Another LLM based logs analysis feature is Root Cause Analysis of an incident where LLM detects the incident based on real time fetched logs and collects the evidence and provides recommendations to resolve the incident.

These features are implemented on a dedicated fastAPI based microservice using RAG (Retrieval Augmented Generation) and LangChain with the help of various LLMs like tiny-llama, DeepSeek-R1(1.5b) and gemini-1.5-pro. In which tiny-llama and deepseek were loaded locally using Ollama and gemini was used with the help of API

Retrieval Augmented Generation (RAG) is a method to leverage applications of LLM based on a custom data to generate response relevant to specific needs. LLM is trained on last public data and its capabilities can be extended using RAG by training it on internal data or knowledge base (logs and metrics in this case). RAG was implemented using Langchain and its components like retriever, prompt builder and QA chain. RAG system architecture demonstrated in Fig 2.

During the implementation the logs from elasticsearch endpoint and index were fetched using elasticsearch client in python language and the logs were embedded in FAISS in memory vectorstore using “all-MiniLM-L6-v2” embedding model available on huggingface. Vectorstores are representations of data in a vector space model which makes data retrieval easier for RAG based pipelines.

Then the LLM was trained on the logs in the vectorstore which was loaded locally using ollama and separate prompt templates were created for each feature with the context of the logs retriever.

Finally API routes were added for both features and separate chains were invoked for each route with trained model and prompt template.

Algorithm:

1. **Initialize Components:** Connect to Elasticsearch, load API key, initialize embedding model, and set up LLM.
2. **Fetch Logs:** Retrieve logs from Elasticsearch, convert them into `Document` objects.
3. **Generate Embeddings:** Use `HuggingFaceEmbeddings` to vectorize logs and store them in FAISS.

4. **Retrieve Relevant Logs:** Query FAISS to find similar logs (`k=logs.size()`).
5. **Apply LLM Prompts:** Use predefined templates for summarization, root cause analysis, and automation mapping.
6. **Execute LLM Queries:** Run `RetrievalQA`, parse structured JSON output.
7. **Store and Update Results:** Maintain `latest_results`, periodically fetch and process logs.
8. **Expose API Endpoints:** Provide FastAPI routes for summary, root cause, and automation retrieval.
9. **Background Processing:** Continuously update logs in separate threads every 3 minutes.

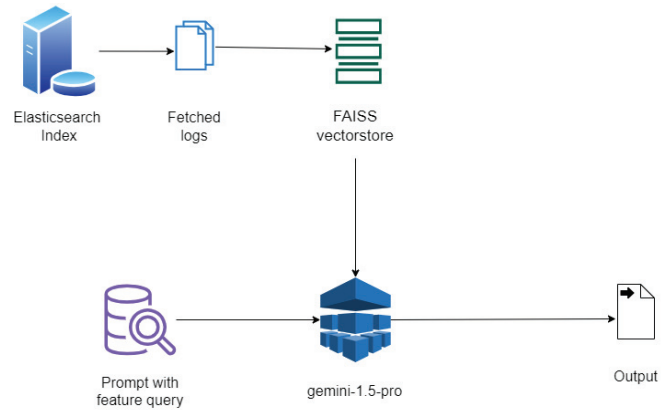


Fig. 2. System architecture of training LLM using RAG

C. Alert generation and self-healing automation

The alerting mechanism in our Logs management Dashboard performs an important role in making sure well timed notifications for critical events. Through leveraging NodeMailer with Node.js, we have carried out an automated alerting machine that detects anomalies and predefined situations in log facts, permitting proactive troubleshooting.

NodeMailer is an open-source email-sending utility used to provide automated email notification. It supports real-time notification by integrating with SMTP to provide timely alert delivery for system crashes within log data.

It makes use of a rules-based technique for flexible alert configurations, along with keyword suits, frequency-primarily based signals, and anomaly detection. NodeJs fetches logs at everyday intervals and identifies problems like mistakes, spikes, authentication screw ups, or system downtime.

Self-healing are the automation tasks which are triggered when an incident occurs where a client can dynamically add mappings of failure and automation tasks (preferably jenkins job) to correct the failure.

Then based on the logs and metrics collected in elasticsearch, the mapping of error and healing job can be passed to the RAG system along with the logs and metrics and then the exact failure can be detected. Then the corresponding automation job is triggered from LogBoard Backend server using Jenkins API.

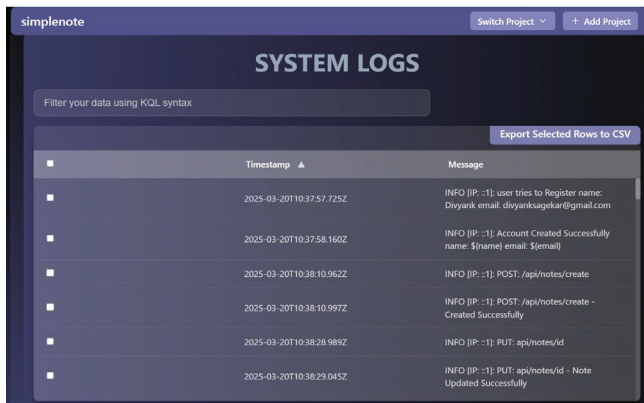
During this implementation 5 default cases were considered shown in Table I:

TABLE I. MAPPING OF FAILURE TYPE AND AUTO-HEALING JOB

Sr.No	Failure Type	Self-Healing Job
1	Application crash	ApplicationRestart
2	ECONNREFUSED	DBRestart
3	Could not connect to database	DBRestart
4	DiskFull	ScaleUp
5	Unauthorized access	BlockSuspiciousIP

IV. RESULTS

Sample logs were collected for a simple-note application which is a MERN stack based note-taking application with CRUD and authentication features. These logs were stored in the elasticsearch index using filebeat on a real time basis. The logs entries shows authentication and note CRUD api logs as shown in Fig 3.



Timestamp	Message
2025-03-20T10:37:57.725Z	INFO [IP:1]: user tries to Register name: Divyank email: divyanksagekar@gmail.com
2025-03-20T10:37:58.160Z	INFO [IP:1]: Account Created Successfully name: \$(name) email: \$(email)
2025-03-20T10:38:10.962Z	INFO [IP:1]: POST: /api/notes/create
2025-03-20T10:38:10.997Z	INFO [IP:1]: POST: /api/notes/create - Created Successfully
2025-03-20T10:38:28.989Z	INFO [IP:1]: PUT: api/notes/id
2025-03-20T10:38:29.045Z	INFO [IP:1]: PUT: api/notes/id - Note Updated Successfully

Fig. 3. Logs saved on elasticsearch

Similarly sample metrics for CPU usage, memory usage and network usage were collected and visualized on LogBoard web interface demonstrated in Fig 4, Fig 5, Fig 6. The trends in these graphs depend on increase and decrease in server load proving systems ability to ingest and process metrics although absolute metrics values are not the focus of the research work..

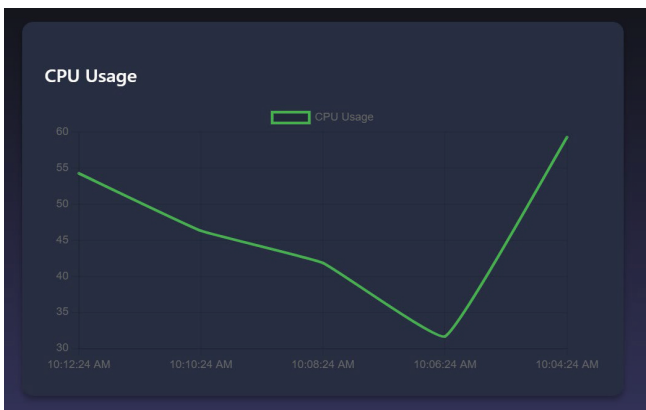


Fig. 4. Metrics saved on elasticsearch (CPU Usage)

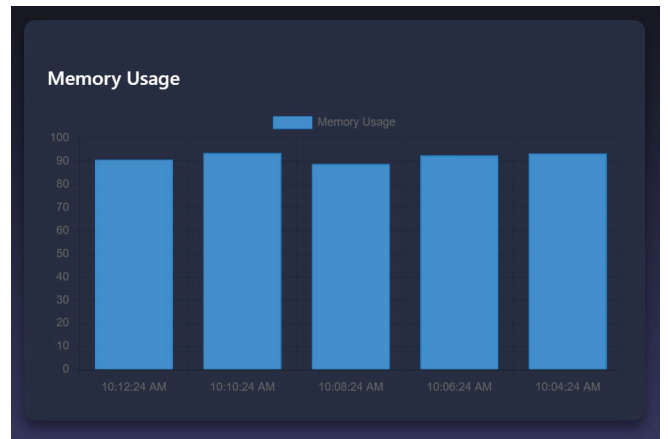


Fig. 5. Metrics saved on elasticsearch (Memory Usage)

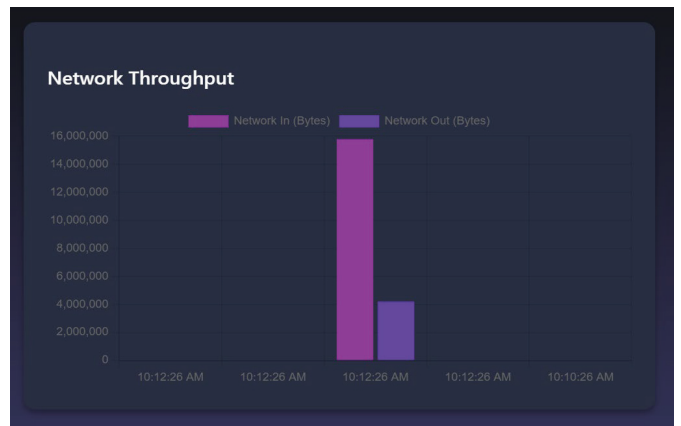
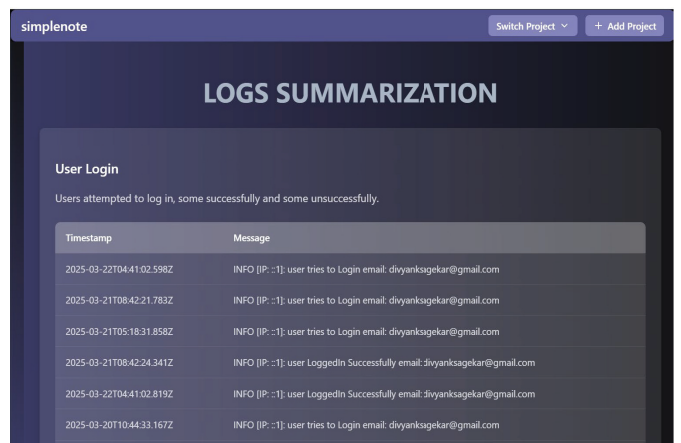


Fig. 6. Metrics saved on elasticsearch (Network Throughput)

For LLM based features the above logs were passed to LLM model and below in Fig 7 is the output for logs summarization and operation wise categorization feature classifying logs into Login, signup, insert update and delete notes categories:



Timestamp	Message
2025-03-22T04:41:02.598Z	INFO [IP:1]: user tries to Login email: divyanksagekar@gmail.com
2025-03-21T08:42:21.783Z	INFO [IP:1]: user tries to Login email: divyanksagekar@gmail.com
2025-03-21T05:18:31.858Z	INFO [IP:1]: user tries to Login email: divyanksagekar@gmail.com
2025-03-21T08:42:24.341Z	INFO [IP:1]: user LoggedIn Successfully email: divyanksagekar@gmail.com
2025-03-22T04:41:02.819Z	INFO [IP:1]: user LoggedIn Successfully email: divyanksagekar@gmail.com
2025-03-20T10:44:33.167Z	INFO [IP:1]: user tries to Login email: divyanksagekar@gmail.com

Fig. 7. Operation Wise logs categorization and summarization

For the feature of Root Cause Analysis the output consists of failure, status, evidence and recommendation for issues like authentication failures and incorrect data provided along with evidences from the ingested logs and recommendation with data correction as shown in Fig 8:

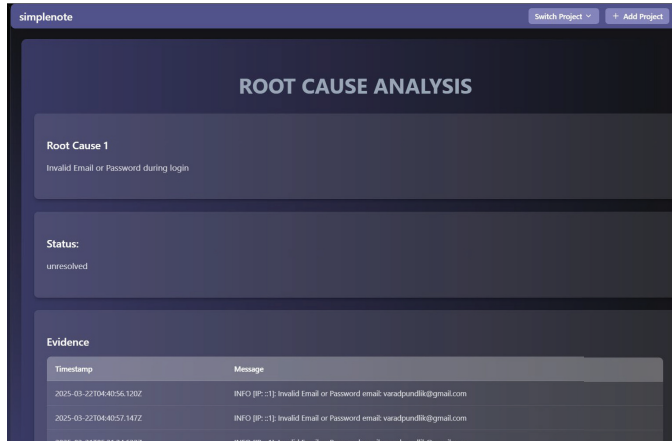


Fig. 8. Root Cause Analysis output

For alert based on logs the web interface shown in Fig 9 was created with email notification for error and critical logs and spike in metrics:

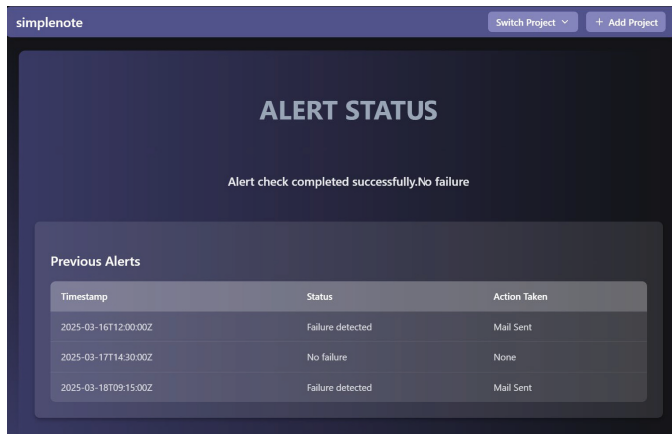


Fig. 9. Alert status output

Based on the failure condition the automation job recommended by LLM service was triggered using jenkins API and output was as below for application restart, db connection restart shown in Fig 10:

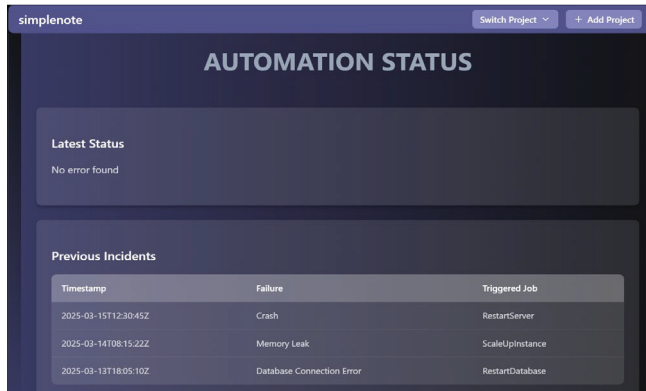


Fig. 10. Automation and self-healing output

All figures in this section use representative logs and metrics from a sample application; the proposed architecture and decision logic remain application-independent and configurable for different production systems.

Performance comparison across various LLMs used in RAG pipeline is as follow as shown in Table II:

TABLE II. PERFORMANCE EVALUATION ACROSS MULTIPLE LLMs

Sr.No	LLM	Interface	Performance Remark
1	tiny-llama	Ollama	Unable to produce output in the provided schema.
2	DeepSeek-r1:1.5b	Ollama	Generated Output in schema but the output was incorrect and generic.
3	gemini-1.5-pro	API	Generated output in the provided schema and output was correct.

The reason for such performance was the first 2 LLMs were running locally and had smaller size whereas gemini was used through API so the larger and faster, efficient model was used.

The generated dashboards, visualizations, and RCA insights demonstrate how the proposed LogBoard system transforms raw operational data into actionable intelligence. From the dashboards and metric charts, teams can instantly detect performance degradation, resource saturation, or abnormal login activity. The operation-wise categorization and LLM-generated summaries enable quick understanding of system behavior without manually parsing complex log files. Root-cause analyses provide contextual explanations and prioritized remedial actions, reducing the time needed for incident triage. Historical OpenSSH case studies show that security anomalies such as brute-force attacks or failed authentications can be automatically identified and correlated with specific hosts or time periods. Through the self-healing module integrated with Jenkins, detected issues can trigger automated recovery workflows like restarting services or blocking malicious IPs. Together, these features enable predictive maintenance, faster resolution, and improved reliability—achieving a shift from reactive monitoring to intelligent, autonomous system observability and operational resilience.

V. CASE STUDY FOR INCIDENT DETECTION ON HISTORICAL LOGS

Apart from application development, operational features like incident detection are crucial for a system in this use case. Hence 2000 historical logs of OpenSSH systems were provided to LogBoard for Root Cause Analysis and detecting incidents from the logs it is also crucial from stress testing perspective to analyze such a large number of logs. The logs consisted of operations like valid or invalid login and logout and logging of authorized and unauthorized access of system. LogBoard detected various root causes and provided their resolutions presented in Table III:

TABLE III. INCIDENTS AND THEIR RESOLUTION OF OPENSSSH SYSTEM

Sr. No	Root Cause	Recommendation
1	Brute-force SSH attacks from various IPs targeting root and other common usernames (admin, user, support, etc.) including invalid user attempts.	Implement stricter SSH security measures. Block or rate-limit suspicious IP addresses. Disable password authentication for root and use key-based authentication. Configure fail2ban to automatically ban IPs after repeated failed login attempts. Consider using a port other than the default 22.
2	Failed reverse DNS lookups for some connecting IPs, indicating possible break-in attempts.	Investigate the IPs with failed reverse DNS lookups. While not conclusive proof of malicious intent, it warrants further scrutiny. Consider blocking these IPs if they exhibit other suspicious behavior.
3	Too many authentication failures leading to disconnections.	This is a consequence of brute-force attempts. Address the root cause of the brute-force attacks as recommended above.
4	Connection reset by peer during SSH authentication from 183.62.140.253.	Check network connectivity between the client and server. This could be a transient network issue or a sign of the client abruptly terminating the connection.

VI. CONCLUSION

This paper presents the development and implementation of a comprehensive logs management dashboard that integrates real-time log monitoring, intelligent log analysis, and automated self-healing mechanisms. Leveraging the ELK stack along with Filebeat and Metricbeat enables scalable and efficient log collection. Additionally, the integration of LLM-based log analysis using a Retrieval-Augmented Generation (RAG) approach facilitates automated log summarization and root cause analysis, significantly improving incident detection and resolution.

The automated alerting system ensures timely notifications for critical events, while the self-healing framework dynamically maps failure patterns to corrective actions, minimizing downtime.

Future work can focus on increasing and measuring performance of RAG based LLM system by passing more

detailed logs and expanding the self-healing mechanisms with more dynamic learning capabilities and improving the accuracy of LLM-based anomaly detection through fine-tuned models. The proposed system lays the foundation for intelligent log management, offering reliability, scalability, and automation for modern distributed systems.

REFERENCES

- [1] Zong, Y.: Distributed log collection for business processes based on ELK architecture. In: IEEE 2nd International Conference on EEBDA, pp. 1523–1529. Changchun, China (2023).
- [2] Okumura, M., et al.: Constructing a log collecting system using Splunk and its application for service support. In: 2016 ACM SIGUCCS Annual Conference (SIGUCCS '16), pp. 103–106. Association for Computing Machinery, New York, NY, USA (2016).
- [3] Chen, T., et al.: Design of log collection architecture based on cloud-native technology. In: 2023 4th Information Communication Technologies Conference (ICTC), pp. 311–315. Nanjing, China (2023).
- [4] Gunasekaran, Raghul et al. Real-Time System Log Monitoring/Analytics Framework. United States, 2011.
- [5] Ramachandran, S., et al.: Automated log classification using deep learning. *Procedia Computer Science* 218, 1722–1732 (2023).
- [6] Setiyadi, Angga et al., Information System Monitoring Access Log Database on Database Server. *IOP Conference Series: Materials Science and Engineering*. 407. 012110. 10.1088/1757-899X/407/1/012110.
- [7] He, S., et al.: A survey on automated log analysis for reliability engineering. *ACM Comput. Surv.* 1(1) (June 2021).
- [8] Chen, Y., et al.: Automatic root cause analysis. In: Nineteenth European Conference Association for Computing Machinery, pp. 674–688. New York, NY, USA (2021).
- [9] Chabridon, S., et al.: Improving performances of log mining. In: IEEE International Symposium on Modeling of Computer and Telecommunication Systems (MASCOTS), pp. 237–243. Milwaukee, WI, USA (2018).
- [10] Lyu, M. R., et al.: Automated log parsing tools. *arXiv preprint* (2019).
- [11] Pereira, G. A., et al.: A log-based mapping study. *arXiv preprint* (2019).
- [11] Akhtar, Siraj, Saad Khan, and Simon Parkinson. "LLM-based event log analysis techniques: A survey." *arXiv preprint arXiv:2502.00677* (2025)
- [12] Guan, Wei, Jian Cao, Shiyu Qian, Jianqi Gao, and Chun Ouyang. "Logllm: Log-based anomaly detection using large language models." *arXiv preprint arXiv:2411.08561* (2024).
- [13] M. Baqar, "Self-Healing Software Systems," *arXiv preprint arXiv:2504.20093*, 2025.
- [14] R. Arora, "AI-Driven Self-Healing Cloud Systems: An Intelligent Framework for Predictive Maintenance and Fault Recovery," *IEEE Access*, vol. 12, pp. 44871–44883, 2024.